



Republic of the Philippines
CARAGA STATE UNIVERSITY
CABADBARAN CAMPUS
T. Curato St., Cabadbaran City, Agusan del Norte
Competence Service Uprightness



COLLEGE OF ENGINEERING AND INFORMATION TECHNOLOGY

LABORATORY MANUAL ACTIVITY

Planning the Flutter Web–REST API–PostgreSQL Technology Stack of Your Own Project

Flutter Framework and Technology Stack

Course	IT 110 – Web Systems and Technologies
Activity	Laboratory Activity No. 3
Output	Flutter Web–REST API–PostgreSQL technology stack blueprint, stack-role worksheets, and group presentation
Maximum Score	80 points

Student / Project Information

Block: <u>A3-IBPH3</u>	Date: <u>September 18, 2026</u>
System Title: RoadWatch: Pothole Detection Using Mobile Dashcam Imagery with a Web-Based Monitoring System	Instructor: Jean Y. Villasin, MIT
Members: RJ Cyrill Jirc L. Monzon; Mae D. Pelipas; Katrice B. Anunciado	

1. Activity Alignment and Purpose

Topic Learning Outcome: Explain the prescribed Flutter Web–REST API–PostgreSQL technology stack and justify the role of each component in the proposed web system.

Course Outcome Alignment: CO1 – Explain web architecture and the roles of client-side, server-side, framework, API, database, and deployment components in the end-to-end flow of a web system.

Syllabus Assessment: Flutter–API–PostgreSQL Technology Stack Blueprint; stack-role explanation; online architecture justification task.

Purpose of the Activity

Activity 2 established the overall architecture of your semester project. In Activity 3, your team will convert that architecture into the prescribed implementation stack: Flutter Web for the client interface, a REST API using an instructor-approved backend framework/runtime for application logic, and PostgreSQL as the persistence layer developed in IT 108. You will also identify the role of Dart, Flutter packages, Git/GitHub, and the deployment environment so the blueprint can guide later implementation activities.

Important distinction: Flutter Web is the client application layer; the REST API is the application and integration layer; PostgreSQL is the persistence layer. The Flutter client must not contain database credentials or issue raw SQL directly to PostgreSQL in the prescribed stack.

2. Intended Learning Objectives

- Distinguish a framework, library/package, runtime, API style/protocol, DBMS, version-control tool, and deployment platform.
- Explain the responsibilities of Flutter Web, the REST API/backend, and PostgreSQL in the team's proposed system.
- Identify the Dart/Flutter fundamentals, widgets, packages, state-management approach, and client-side validation needed by major project features.
- Select and justify an instructor-approved backend framework/runtime for the REST API without shifting database or client responsibilities into the wrong layer.
- Map REST API resources and operations to the PostgreSQL tables, relationships, constraints, and transactions being developed in IT 108.
- Create a system-specific Flutter Web–REST API–PostgreSQL technology stack blueprint that includes Git/GitHub, security boundaries, and deployment targets.
- Explain and defend why each technology belongs in the stack and how the components communicate.

3. Activity Specifications

Activity Item	Requirement
Mode	Existing semester-project group; all members must contribute to the technology decisions and be able to explain one assigned stack component.
Suggested Time	One laboratory session, approximately 3 hours, plus the scheduled group presentation.
Primary Output	One project-specific Flutter Web–REST API–PostgreSQL Technology Stack Blueprint.

Activity Item	Requirement
Supporting Output	Technology classification worksheet, stack-role matrix, Flutter client plan, REST API plan, PostgreSQL mapping, Git/GitHub and deployment plan, 120–180 word stack-role explanation, and group presentation.
Recommended Tools	diagrams.net/Draw.io, Lucidchart, Figma, Canva, PowerPoint, Git/GitHub, Flutter/Dart documentation, backend API documentation, and PostgreSQL documentation.
Presentation Format	5–7 minutes per group followed by 2–3 minutes of questions. Every member must explain an assigned technology, interface, or integration decision.

4. Pre-Laboratory Requirements

- Final or revised architecture diagram from Laboratory Activity No. 2.
- Approved system title, primary users/roles, and at least five major system features.
- Current ERD or normalized PostgreSQL schema from IT 108, including primary keys, foreign keys, and important constraints.
- Instructor-approved backend framework/runtime for the REST API. If approval is still pending, bring no more than two candidate options for consultation.
- Initial list of Flutter packages/state-management tools and external services that the project genuinely needs.
- Access to the project GitHub repository and the team's planned deployment environment or hosting options.
- Copies of Flutter/Dart, REST API/backend, and PostgreSQL documentation relevant to the selected technologies.

5. Part A – Technology Classification and Role Check

Before assigning technologies to the project, classify each item correctly. Use the following distinctions: **framework** provides an application structure and conventions; **library/package** provides reusable functions or components; **runtime** executes application code; and the remaining categories describe communication, persistence, version control, or hosting responsibilities.

Category	Meaning
Framework	Provides structure, conventions, lifecycle, and reusable building blocks for an application.
Library / package	Reusable code added to the application for a focused capability such as HTTP, state management, charts, maps, or validation.
Runtime	Environment that executes application or server code.
API style / protocol / data format	Defines how components communicate and how requests/responses are represented.
DBMS	Manages persistent structured data, constraints, transactions, and queries.
Version control / collaboration	Tracks changes, supports branches and reviews, and stores project history.
Deployment platform / environment	Runs or hosts the client, API, database, and supporting services.

Classification Worksheet

Technology / Concept	Classification	Role in Your Project	Project-Specific Choice / Evidence
Dart	Programming language	Defines Flutter application logic	Dart 3.13.0
Flutter	Framework	Builds responsive monitoring UI	Flutter Web 3.47.0
Flutter package for HTTP/API calls	Library / package	Sends requests and parses JSON	http
State-management package or pattern	Pattern / package	Keeps loading, error and data state	Provider / ChangeNotifier
REST + HTTPS + JSON	API style / protocol / format	Client-server communication	REST + HTTPS + JSON
Backend language	Language	Implements API and business rules	Python
Backend runtime	Runtime	Executes FastAPI application	Python 3; Uvicorn ASGI server
PostgreSQL	DBMS	Stores validated pothole records	PostgreSQL 18
Git / GitHub	Version control	Tracks and reviews team changes	Git + GitHub
Deployment platform(s)	Deployment environment	Hosts web, API and database	Flutter Web host + API server + PostgreSQL

Flutter Web dashboard and its listed packages are proposed for the course; the current web dashboard uses HTML/JavaScript.

6. Part B – Prescribed Stack Role Matrix

Complete the matrix using the exact technologies planned for your system. The “Must Not Be Responsible For” column is included to prevent duplicated logic and insecure shortcuts.

Layer / Area	Prescribed Technology	Your Project-Specific Choice	Main Responsibility	Must Not Be Responsible For
Presentation / Client	Flutter Web + Dart	Flutter Web 3.47.0 + Dart 3.13.0	Screens, widgets, navigation, state, form handling, user feedback	Database credentials; raw SQL; final authorization rules
Client packages	Team-selected Flutter packages	http, provider, go_router, fl_chart, flutter_map	HTTP/API calls, state management, validation, maps/charts, or other approved client functions	Server-only secrets or database administration
Integration	REST API over HTTPS using JSON	REST API over HTTPS using JSON	Defines endpoints, request/response formats, status codes, and client-server communication	Persistent data storage by itself
Application / Backend	FAST API / Python	FastAPI + Python; Uvicorn; SQLAlchemy	Authentication/authorization checks, validation, business rules, orchestration, API response generation	Rendering the Flutter UI; storing production data in local code files
Persistence	PostgreSQL schema developed in IT 108	PostgreSQL 18 RoadWatch schema	Tables, relationships, constraints, transactions, queries, durable records	User-interface logic or route navigation
Version control	Git + GitHub	Git + private/public GitHub group repository	Source history, collaboration, branches, pull requests, issue/document tracking	Running the production system
Deployment	Team-selected hosting / server / managed DB	Web host; FastAPI server/container; PostgreSQL server	Runs Flutter Web, API, PostgreSQL, and environment configuration	Replacing application-layer security or database constraints

Stack-role rule

A good blueprint separates concerns. Flutter Web handles the user experience and client state; the REST API/backend protects and processes requests; PostgreSQL persists validated data and enforces relational integrity. Git/GitHub supports collaboration, while deployment platforms host the running components. If a responsibility appears in two layers, the team must explain why it is not duplicated or contradictory.

7. Part C – Flutter Web Client Blueprint

Plan the client side around your actual system features. List at least five important screens or feature modules. Use package names only when the team has decided to use them.

Feature / Screen	Main Widgets / Forms	State / Controller	Flutter Package(s)	API Operation Needed	Client Validation / User Feedback
Login	Form, TextFields, button	AuthController	http, provider	POST /auth/login	Required fields; invalid-login error
Dashboard	Cards, counters, chart	DashboardController	fl_chart, http	GET /detections	Aggregate returned records; loading/error states
Pothole Map	Map, markers, filters	DetectionController	flutter_map, latlong2	GET /detections	Validate date/status filters
Detection List	DataTable, search, paging	DetectionController	http, provider	GET /detections	Search feedback; retry on failure
Detection Detail	Image, coordinates, status form	DetectionController	http	GET/PATCH /detections/{id}	Valid status; confirm update

Client planning checklist:

- Routing/navigation between screens is identified.
- Responsive web layout and loading/error/empty states are considered.
- Client-side validation improves usability but does not replace server-side validation.
- No database password, private API key, or other server secret is embedded in Flutter Web source code.

8. Part D – REST API and Backend Plan

Selected backend framework: FastAPI

Selected backend runtime: Python 3; Uvicorn ASGI server

Why this choice fits the project (3–5 sentences):

FastAPI matches the existing RoadWatch backend and provides typed validation, clear REST routes, and automatic API documentation. Python integrates with SQLAlchemy and PostgreSQL. Python 3 executes the backend code, while Uvicorn serves the ASGI application. Server-side rules keep mobile and web clients consistent.

Minimum API Contract Worksheet

Define at least six REST operations that support the project's real workflows. Use an instructor-approved endpoint naming convention and appropriate HTTP methods.

Use Case	Method	Endpoint	Auth / Role	Request Data	Server Action / Rule	Response / Status
Sign in	POST	/auth/login	Public	email, password	Verify account; issue token	200/401 JSON

Use Case	Method	Endpoint	Auth / Role	Request Data	Server Action / Rule	Response / Status
List potholes	GET	/detections	Authorized	page/filter	Filter and paginate	200 JSON
Create detection	POST	/detections	Mobile token	class=3, confidence, GPS, time, client_event_id	Validate; reject duplicate	201/409/422
View detection	GET	/detections/{id}	Authorized	path id	Return matching record	200/404
Update status	PATCH	/detections/{id}	Admin	status, notes	Validate transition; audit	200/403/422
Delete/Archive	DELETE	/detections/{id}	Admin	path id	Soft archive; preserve history	204/403/404

REST API guidance

The API should validate input, verify identity and role permissions, apply business rules, interact with PostgreSQL or approved external services, and return an appropriate HTTP status with JSON data. Common methods include GET, POST, PUT/PATCH, and DELETE. The exact endpoints and rules must come from your own project requirements.

9. Part E – PostgreSQL Persistence and IT 108 Integration

Map the API resources to the PostgreSQL schema being designed in IT 108. Do not invent a second unrelated database model for IT 110. Record important keys, constraints, and transaction rules that affect the API.

API Resource / Feature	PostgreSQL Table(s)	PK / FK / Constraint	DML Operation	Transaction / Integrity Rule	Returned Fields / Result
Authentication	users	user_id PK; email UNIQUE; role CHECK	SELECT	Only active users authenticate	token, role
Pothole records	detections	detection_id PK; user_id FK; client_event_id UNIQUE	INSERT	One transaction; class must be Pothole	id, confidence, GPS, time
Detection list/map	detections	valid latitude/longitude; confidence 0..1	SELECT	Parameterized filters and pagination	records + total
Status update	detections, audit_logs	detection_id FK; status CHECK	UPDATE + INSERT	Atomic update and audit entry	new status, updated_at
Offline sync	detections	client_event_id UNIQUE	UPSERT/INSERT	Idempotent retry prevents duplicates	created/existing id

Database boundary rule

In the prescribed Flutter Web–REST API–PostgreSQL stack, Flutter Web should not connect directly to PostgreSQL. The backend owns database credentials and uses the database driver/ORM or approved data-access layer. PostgreSQL remains responsible for persistent records, keys, constraints, transactions, and query execution.

10. Part F – Git/GitHub and Deployment Plan

Planning Item	What Must Be Decided	Your Project Decision / Evidence
Repository ownership	Group repository URL, members with access, and repository visibility as instructed	Team-owned GitHub repository; all three members have access; URL added after repository creation.
Folder structure	Identify where Flutter client, backend API, documentation, and database scripts/migrations will be stored	mobile/ Flutter dashcam; dashboard/ Flutter Web; api/ FastAPI; database/ migrations; ml/ model metadata; docs/technology-stack/.
Branching / collaboration	Main branch, feature branches, commit/merge practice, and review responsibility	main is protected; feature/* branches; descriptive commits; another member reviews before merge.
Environment variables	List values that must be configured outside source code, such as API base URL and server/database secrets	API_BASE_URL on client; DATABASE_URL, JWT_SECRET and CORS_ORIGINS on server. Real secrets excluded; .env.example lists names only.
Flutter Web deployment	Hosting platform or server for the built web client	Build with flutter build web; deploy build/web to an approved HTTPS static host.
REST API deployment	Server, container, cloud runtime, or approved hosting platform	Deploy FastAPI/Uvicorn on an approved Python server or container with HTTPS reverse proxy.
PostgreSQL deployment	Local development instance and planned managed/server production database	PostgreSQL 18 locally for development; managed/server PostgreSQL for production with restricted network access.
Logging / backup / recovery	Minimum plan for logs, error diagnosis, database backup, and restoration	Structured API logs without secrets; health/error monitoring; scheduled pg_dump backups; test restoration and document recovery steps.

Recommended minimum repository organization (adapt as needed):

```
project-root/
  frontend/      Flutter Web application
  backend/       REST API source code
  database/      PostgreSQL scripts or migrations, when used
  docs/          architecture and technology-stack blueprints
  README.md
  .gitignore
  .env.example   variable names only; never commit real secrets
```

11. Part G – Construct the Technology Stack Blueprint

Create one clean, readable blueprint that shows how the prescribed stack is implemented in your own system. The blueprint must be understandable without opening the source code.

No.	Required Element	What Must Be Shown
1	System identity	Official project title and "Flutter Web–REST API–PostgreSQL" stack label.
2	Users / actors	Major users who interact with the Flutter Web client.
3	Flutter Web client	Dart, key screens/widgets, routing, state management, form validation, and HTTP/API client.

No.	Required Element	What Must Be Shown
4	Flutter packages	Only packages actually planned for the project; state their purpose.
5	Communication	HTTPS, REST endpoints, JSON request/response, and API base URL/environment configuration.
6	Backend framework/runtime	Exact instructor-approved framework and runtime used to implement the API.
7	Application logic	Authentication, authorization, validation, services, business rules, and external-service integration.
8	PostgreSQL persistence	Relevant tables/data domains from IT 108, keys/constraints, and database access mechanism.
9	Git/GitHub	Repository and collaboration boundary for client, backend, database scripts, and documentation.
10	Deployment	Where the Flutter build, REST API, PostgreSQL database, and storage/external services will run.
11	Security boundary	Show where secrets live, where tokens/roles are checked, and which resources are protected.
12	Data flow	Directional arrows labeled with requests, responses, JSON, SQL/data access, and returned results.

Minimum Blueprint Rule

Your blueprint must show at least one complete implementation path: User → Flutter Web interface → HTTPS/JSON REST request → backend API logic → PostgreSQL transaction/query → JSON response → Flutter state/UI update. Include Git/GitHub and deployment information as supporting infrastructure, not as steps in the runtime request path.

12. Reference Example – Do Not Copy as Your Final Output

The figure below shows the level of stack separation and labeling expected. Replace every generic label with the technologies, features, endpoints, and database objects used by your own project.

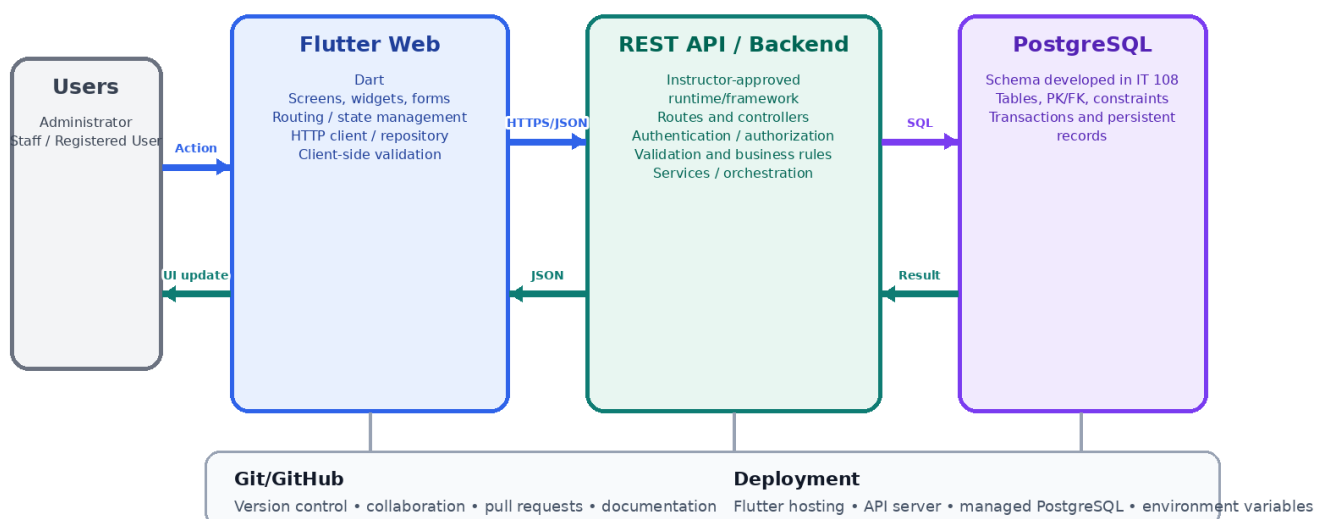
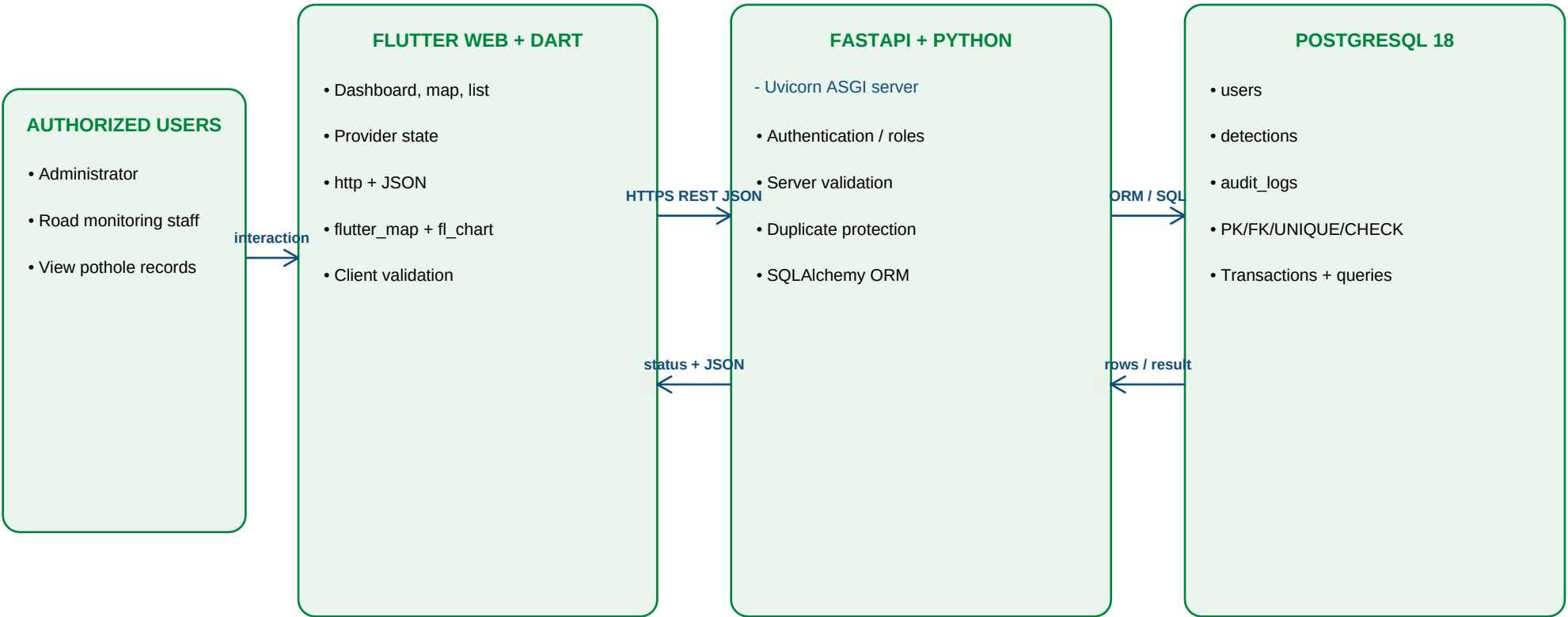


Figure 1. Reference Flutter Web–REST API–PostgreSQL technology stack and runtime path.

RoadWatch Technology Stack Blueprint

Pothole Detection Only | Flutter Web - REST API - PostgreSQL

Proposed course implementation: Flutter Web dashboard. The current RoadWatch dashboard uses HTML/JavaScript. The existing Flutter mobile app, FastAPI backend and PostgreSQL database remain the basis of this plan.



Complete path: User → Flutter UI → HTTPS/JSON → FastAPI rules → PostgreSQL transaction/query → JSON → Flutter state/UI

SECURITY AND CONFIGURATION BOUNDARY

Client: API_BASE_URL only | Server: DATABASE_URL, JWT_SECRET, CORS_ORIGINS
Tokens and roles checked in FastAPI | Database credentials never stored in Flutter Web

SUPPORTING INFRASTRUCTURE

Git + GitHub: mobile/, dashboard/, api/, database/, docs/ | Deployment: HTTPS web host + FastAPI server/container + restricted PostgreSQL server

Step	Reference Flow
1	User interacts with a Flutter Web screen.
2	Flutter validates the form and sends an HTTPS request containing JSON.
3	The REST API verifies access, validates data, and executes business rules.
4	The backend queries or updates PostgreSQL using an approved data-access mechanism.
5	The API returns an HTTP status and JSON payload.
6	Flutter updates state and displays the result to the user.

13. Part H – Feature-to-Stack Responsibility Trace

Selected workflow / feature: Review and monitor a synchronized pothole detection

Expected result shown to the user:

Authorized dashboard user sees the validated pothole marker, details, and current status.

Step	Responsible Component	Action / Data	Why It Belongs in This Component	Evidence in Blueprint
1	Flutter Web	User opens Pothole Map	Owens presentation and interaction	Map screen
2	Flutter Web	Validate filters; GET /detections	Client validation and HTTPS request	HTTPS/JSON arrow
3	FastAPI	Verify token/role and filters	Server owns access and business rules	API logic box
4	SQLAlchemy + PostgreSQL	Query valid pothole rows	Backend data access; DB integrity	SQL/data arrow
5	FastAPI	Return 200 and JSON records	API creates standard response	JSON response arrow
6	Flutter Web	Update state and show markers	Client renders returned data	State/UI update

14. Part I – Stack-Role Explanation

Write 120–180 words explaining and justifying the prescribed technology stack for your own system. Address all of the following:

- What Flutter Web and Dart are responsible for in the user experience.
- Why the REST API/backend is the correct place for authentication/authorization checks, server-side validation, and business rules.
- How PostgreSQL persists the data model being developed in IT 108 and protects relational integrity.
- Why Git/GitHub and the deployment environment are necessary even though they are not runtime application layers.
- How the separation improves security, maintainability, testing, integration, and future scaling, including one implementation risk the team must manage.

RoadWatch proposes Flutter Web and Dart for the monitoring dashboard, input validation, client state, and HTTPS requests.

The FastAPI backend is the proper location for authentication, authorization, server-side validation, duplicate protection, and status rules because these controls must apply consistently to both mobile and web clients. SQLAlchemy accesses PostgreSQL 18, where primary keys, foreign keys, unique client event IDs, checks, and transactions protect the IT 108 data model and preserve reliable pothole records. Git and GitHub support version history, branches,

reviews, and team collaboration, while deployment environments run the built Flutter client, API, and database with secrets stored outside source code. This separation improves security, maintainability, testing, integration, and future scaling. A key implementation risk is unreliable mobile connectivity, which may cause repeated sync attempts; idempotent event IDs, an offline queue, and transactional inserts will prevent duplicate pothole records.

15. Final Quality Checklist

Checklist A	Checklist B
☐ The official system title and actual project features are used.	☐ The stack is explicitly labeled Flutter Web–REST API–PostgreSQL.
☐ Framework, library/package, runtime, DBMS, version control, and deployment platform are classified correctly.	☐ Flutter screens/widgets, client state, packages, and API calls are project-specific.
☐ The instructor-approved backend framework/runtime is named.	☐ At least six meaningful REST operations are defined.
☐ Authentication, authorization, validation, and business rules are located primarily in the backend/API layer.	☐ The PostgreSQL mapping matches the IT 108 schema or clearly identifies coordinated schema changes.
☐ Flutter Web does not contain database credentials or direct raw-SQL access to PostgreSQL.	☐ HTTPS/REST/JSON and database-access arrows are directional and meaningfully labeled.
☐ Git/GitHub organization, environment variables, and deployment targets are identified.	☐ At least one complete User → Flutter → API → PostgreSQL → API → Flutter path is visible.
☐ The written stack-role explanation agrees with the blueprint and worksheets.	☐ Every group member can explain one component and defend the complete integration path without reading a script.

16. Submission Requirements

- PDF or PNG of the final technology stack blueprint in readable landscape orientation.
- Editable blueprint source file such as .drawio, .pptx, .fig, or the native file of the selected diagramming tool.
- Completed Parts A–H and the 120–180 word stack-role explanation.
- Updated project GitHub repository containing the blueprint and supporting files under a folder such as docs/technology-stack/.
- Repository link and required files submitted through MasaoLMS using the naming convention IT110_LA3_SystemName.
- Deliver a 5–7 minute group presentation using the final blueprint as the main visual, followed by 2–3 minutes of questions. Every member must participate meaningfully.

Academic Integrity and Competency Validation

The output must represent the team's own system and technology decisions. AI tools may be used to clarify terminology or review ideas, but the final stack classification, framework/runtime selection, endpoint plan, PostgreSQL mapping, blueprint, and justification must be verified, understood, and defended by the team. The instructor may ask any member to explain why a component belongs in a particular layer or trace a request without assistance.

17. Scoring Rubric – 80 Points

Partial credit may be awarded when evidence is present but incomplete, inconsistent, or technically inaccurate. For the presentation criterion, points may be reduced for unequal participation, excessive reading, incorrect role assignment, weak integration reasoning, or inability to answer questions about the team's own stack.

Criterion	Evidence for Full Credit	Points
-----------	--------------------------	--------

Criterion	Evidence for Full Credit	Points
Framework/library/runtime and stack-role accuracy	Technologies are classified correctly, and the team clearly distinguishes Flutter, Dart, packages, REST, backend runtime/framework, PostgreSQL, Git/GitHub, and deployment responsibilities.	10
Flutter Web client planning	Screens/features, widgets/forms, state approach, packages, client validation, API calls, loading/error feedback, and client security boundaries are complete and project-specific.	10
REST API and backend plan	An instructor-approved backend framework/runtime is justified; endpoints, methods, authentication/roles, validation, business rules, request data, responses, and status behavior are coherent.	14
PostgreSQL persistence integration	API resources map correctly to the IT 108 schema; important tables, PK/FK constraints, DML operations, transactions, and integrity requirements are represented accurately.	11
Technology stack blueprint and end-to-end integration	The blueprint clearly integrates users, Flutter Web, packages/state, HTTPS/REST/JSON, backend application logic, PostgreSQL, security, and a complete request-response path.	15
Git/GitHub, deployment, and secret management	Repository organization, collaboration approach, environment variables, hosting targets, and protection of server/database secrets are identified and realistic.	5
Written stack-role explanation	The 120–180 word explanation accurately justifies component roles, separation of concerns, IT 108 integration, security, maintainability, testing, deployment, scaling, and one implementation risk.	5
Group presentation and oral defense	Presentation is clear, organized, within time, and uses the final blueprint effectively; all members participate and correctly defend technology roles, API/database boundaries, and the selected workflow.	10

Raw Score	Equivalent	Checked by	Date
/ 80	_____	_____	_____

18. Post-Laboratory Reflection

Which responsibility was hardest for your group to assign correctly among Flutter Web, the REST API, and PostgreSQL, and why?

The hardest was separating duplicate protection: Flutter queues retries, but FastAPI and PostgreSQL must make creation idempotent.

Which Flutter package or client-side pattern is most important to your project, and what problem will it solve?

Provider/ChangeNotifier is most important because it keeps map, list, loading, sync, and error states consistent.

What server-side rule must never rely only on Flutter client validation?

Pothole class, confidence range, GPS range, authorization, and duplicate-event checks must always be enforced by FastAPI.

Which PostgreSQL constraint or transaction rule from IT 108 is most important to preserving correct system data?

A UNIQUE client_event_id plus an atomic insert/status transaction best prevents duplicate or partially updated records.

What must your team configure before the first real Flutter-to-REST-API integration test can succeed?

Set the correct API_BASE_URL, CORS origins, HTTPS/network access, test account/token, and PostgreSQL connection variables.

What is the first stack component your group should implement after this activity, and what evidence will prove that it works?

Implement and test POST /detections first; evidence is a 201 JSON response and one matching PostgreSQL row after a retry.